GENERATIVE AI ASSIGNMENT · PROJECT REPORT

Plum 360 Assistant

A product guide and retrieval-augmented chat assistant for the Plum 360 infusion pump, grounded in its 268-page System Operating Manual. Every answer cites the manual pages it came from, and those pages open inside the app.

PROJECT TYPE
Retrieval-augmented generation (RAG) agent with a web front end

SOURCE DOCUMENT
Plum 360 System Operating Manual, IFU0000340 (2021-10), 268 pages

CORE STACK
Python 3.12 · LangGraph · Chroma · Groq · FastAPI · React

STATUS
Working end to end; 65 automated tests pass; 24 of 26 evaluation questions pass

1 Overview and objectives
2 System architecture
3 Data pipeline: from PDF to index
4 Retrieval: hybrid search
5 Agent workflow (LangGraph)
6 Language models and providers
7 Safety and grounding
8 Backend API
9 Front end
10 Evaluation
11 Automated tests
12 Technology stack
13 Repository structure
14 Design decisions and problems solved
15 Limitations and future work
16 How to run
17 Conclusion

1. Overview and objectives

Infusion pumps ship with long operating manuals that clinicians and biomedical technicians rarely have time to read in full. The Plum 360 Assistant turns one such manual into an interactive reference: a product guide that summarises the device, and a chatbot

that answers free-text questions by searching the manual, summarising the relevant passages with a large language model (LLM), and citing the exact pages used.

The project was built to demonstrate a complete generative AI application rather than a single model call. It covers document processing, vector and keyword retrieval, an agent workflow with routing and verification, streaming answers, a safety layer appropriate to a medical-device subject, a production-style API, a React user interface, automated tests, and a measured evaluation.

Objectives

1. **Ground every answer in the manual.** The model may only summarise retrieved passages; device facts are never taken from its general knowledge.
2. **Make answers verifiable.** Each answer lists its source pages, and the user can open the rendered page to check.
3. **Behave like a chatbot.** Greetings, thanks, and follow-ups such as "explain that more simply" are handled conversationally; device questions trigger a manual search.
4. **Refuse unsafe requests.** Requests to program the pump, change settings, or dose a patient are refused before reaching the model.
5. **Measure quality.** An evaluation set checks routing, retrieval, factual content, scope, and safety end to end.

268	103	92%	65	~1–3 s
manual pages processed	indexed text chunks	evaluation pass rate (24/26)	automated tests passing	typical answer time

> **Scope statement.** This software is an independent reference project. It is not a medical device, is not affiliated with the manufacturer, and must not be used for dosing, programming, or alarm-response decisions. This statement appears on every screen of the application.

---

2. System architecture

The system has three tiers. A **React front end** presents the product guide and the chat. A **FastAPI backend** exposes the assistant and the manual pages over HTTP. The **assistant core**, a Python package named `plum360_assistant`, contains the agent workflow, retrieval, safety checks, and model clients. The core is shared by the API and by a Streamlit prototype, so both behave identically.

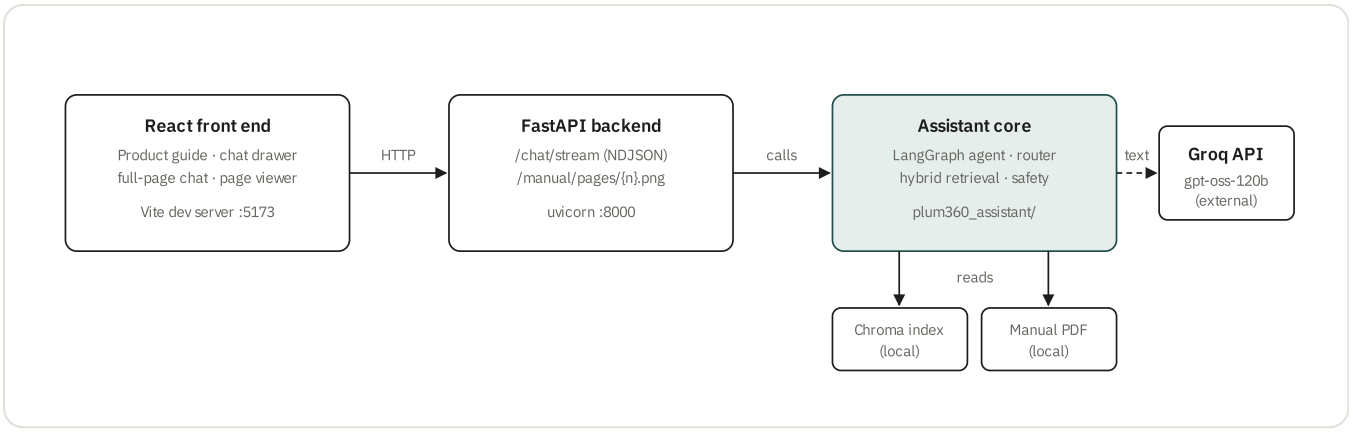


Figure 1. Deployment view. Only the question and the retrieved excerpts leave the machine (to Groq for text generation); the manual, its index, and the embedding model all stay local.

---

3. Data pipeline: from PDF to index

The manual is processed once, offline, by four command-line scripts. Each step writes a file that the next step reads, so every stage can be inspected and re-run on its own.

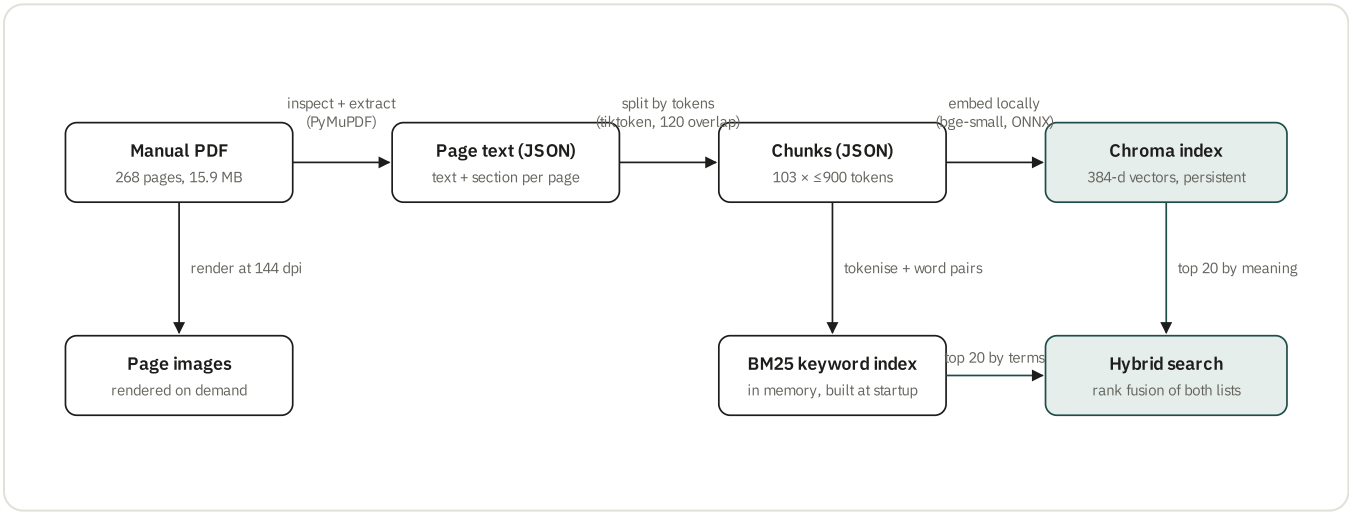


Figure 2. Indexing pipeline. The top row runs once offline; the bottom row is built or rendered at runtime from the same artefacts.

3.1 Inspection (`scripts/inspect_documents.py`)

Before extraction, the PDF is opened with PyMuPDF and checked page by page. The inspector records the document metadata (title, producer, page count), flags pages with very little text, and reports extraction errors. For this manual, five pages were flagged as low-text (mostly blank "Notes" pages) and no errors occurred. The report is saved as JSON so the decision to proceed is documented.

3.2 Extraction (`scripts/extract_documents.py`)

Each page's text is extracted with its page number and the section heading it falls under, producing `data/processed/plum360_manual.json`. Keeping the page boundary at this stage is what later makes page-level citations possible.

3.3 Chunking (`scripts/chunk_documents.py`)

Pages are joined and split with LangChain's `RecursiveCharacterTextSplitter`, measured in tokens with `tiktoken`. The configuration is 900 tokens per chunk with 120 tokens of overlap, which yields 103 chunks. Repeated header and footer lines (the manual's running titles) are detected and chunks that contain nothing else are dropped. Each chunk records every page it overlaps, so a chunk that straddles pages 182 to 184 cites all three.

3.4 Embedding and indexing (`scripts/index_chunks.py`)

Chunks are embedded and stored in a persistent Chroma collection under `data/chroma_db/`. Indexing is idempotent: a content hash per chunk means re-running the script adds, updates, or deletes only what changed. Two embedding providers are supported, each with its own collection so vector dimensions never mix:

- **Local (default):** `BAAI/bge-small-en-v1.5` through `fastembed`, a 384-dimension model that runs on the CPU with ONNX. It downloads once (about 130 MB) and needs no API key or quota; manual text never leaves the machine.
- **Gemini (optional):** `gemini-embedding-001` (3072 dimensions) through the Gemini API, with paced batches to respect free-tier limits.

3.5 Page rendering and figures (`plum360_assistant/manual_pages.py`)

The same PDF is used at runtime. Any page can be rendered to PNG at 144 dpi for the in-app viewer, and the manual's printed page labels (for example `11-6`) are mapped to PDF page numbers by scanning each page's running header and footer; 254 of 268 pages carry a label. A separate script crops four equipment diagrams from the manual for the product guide. All of this output is git-ignored because it contains copyrighted material.

---

4. Retrieval: hybrid search

Vector search alone finds passages with similar meaning but can rank exact manual terms too low. When the local embedding model was first tested, it found the correct page in the top three results for only 9 of 16 probe questions. Two complementary rankings are therefore fused:

1. **Dense retrieval.** The question is embedded (with the BGE query instruction prefix) and the 20 nearest chunks are fetched from Chroma by cosine distance.

2. **Sparse retrieval.** A small BM25 index (k1 = 1.4, b = 0.75) is built in memory over the same chunks at startup. Tokens are lower-cased, stop-words removed, plurals normalised, and adjacent word pairs added as extra terms, so phrases such as *air trap* or *rate range* score as rare, specific terms.
3. **Fusion.** The two ranked lists are merged with reciprocal rank fusion (k = 60). One refinement proved important: the top item of each list is always kept at the front, because a chunk that is the single best match by meaning or by exact terms is too strong a signal to be averaged away by a poor rank in the other list.

RETRIEVAL CONFIGURATION (LOCAL EMBEDDINGS)	CORRECT PAGE IN TOP 3
Vector search only	9 / 16
Vector + BM25 with rank fusion	14 / 16
Vector + BM25 with word-pair terms (wider probe set)	18 / 20

Smaller chunks (400 tokens) and a long-context embedding model (`jina-embeddings-v2-small-en`) were also tried; neither improved on this configuration, so the 900-token chunks were kept.

---

5. Agent workflow (LangGraph)

The assistant is a LangGraph `StateGraph`. Each node reads and updates a typed state (question, history, retrieved chunks, relevance verdict, draft answer, citations, retry count). Conditional edges decide the path, so a greeting never touches the index and a device question never skips verification.

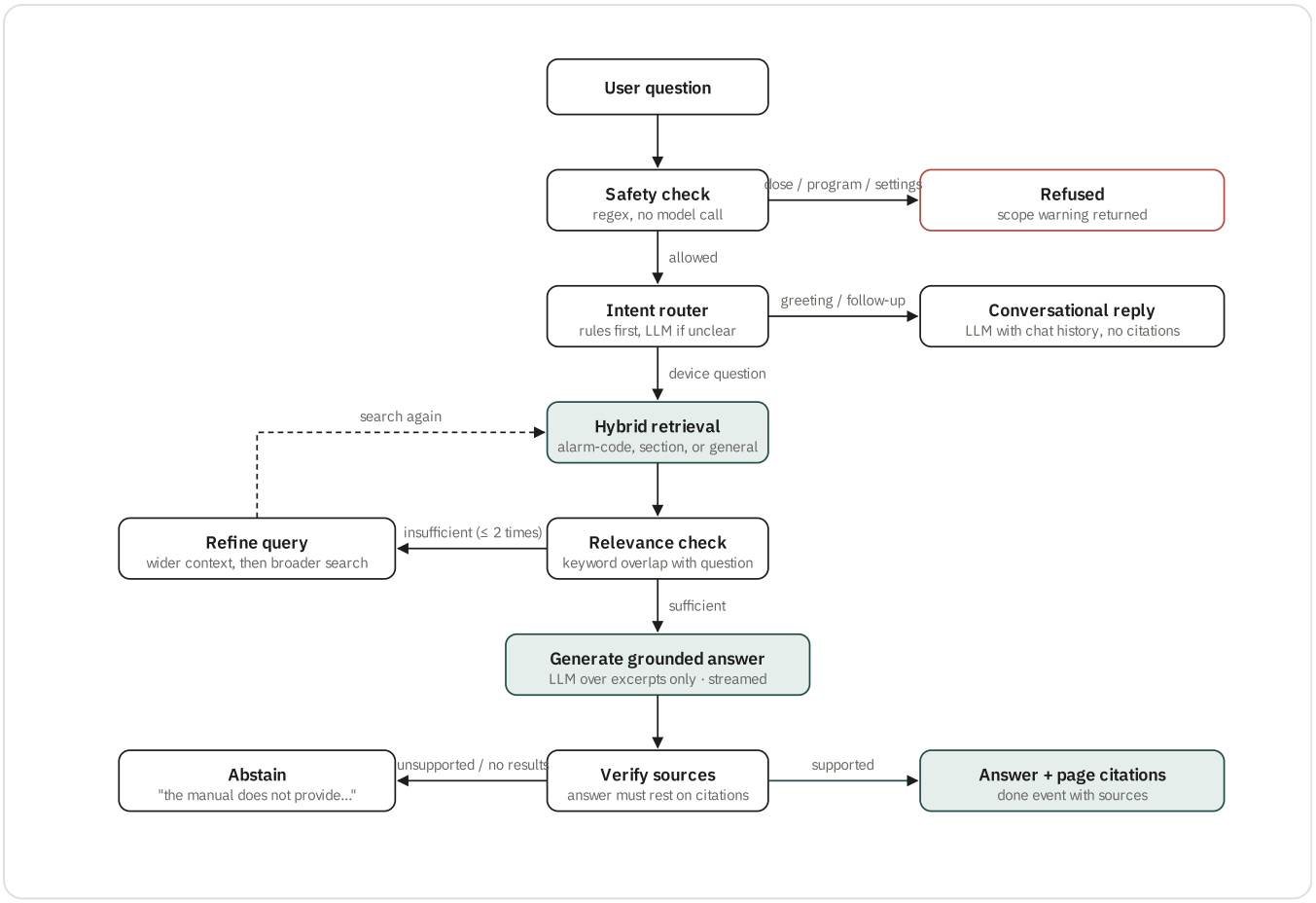


Figure 3. Query pipeline. Shaded nodes make a model or index call; the others are deterministic Python. Status events ("Searching the manual", "Reading the relevant pages") are emitted as the graph moves between nodes.

NODE	WHAT IT DOES
`receive_question`	Normalises the question and resets per-turn state; keeps the conversation history passed by the client.
`route_intent`	Classifies the message as *conversation* or *manual*. Obvious cases (greetings, thanks, "explain that", alarm codes, device vocabulary) are decided by regular expressions with no model call; unclear messages are classified by the LLM with a one-word prompt. On any failure the manual path is chosen, as the safe default.
`respond_conversationally`	Calls the LLM with a system prompt and the role-tagged history. The prompt forbids answering device or clinical questions from general knowledge and limits rephrasing to the previous answer's content.
`analyze_question` / `select_tool`	Picks a retrieval tool: `search_by_alarm_code` when a code such as E301 is present, `search_by_section` when a section is named, otherwise `search_manual`.
`retrieve_manual_information`	Runs the chosen tool (top 3 chunks). For alarm codes the evidence is narrowed to the lines around that exact code so neighbouring alarm entries do not leak in.
`assess_relevance`	Marks results insufficient when no chunk shares a keyword with the question.
`refine_search`	On insufficient results, retries once with wider document context and once with a broader query (top 5). At most two retries.
`generate_grounded_answer`	Prompts the LLM to summarise the excerpts in 2–4 sentences. Alarm answers use a fixed two-line format (*Meaning* / *Manual-listed action*), and the listed corrective actions are completed from the source text so none are dropped. Output streams token by token.
`verify_answer`	Checks that the answer rests on manual evidence and carries citations.
`revise_or_abstain`	Replaces an unsupported answer with an explicit abstention that points the user to the official manual.

6. Language models and providers

Text generation uses the Groq API, chosen for its free tier (about 1,000 requests per day) and very low latency. Models are tried in order; any failure (overload, rate limit, timeout, retired model) moves to the next one through LangChain's `with_fallbacks`. Gemini chat models can be appended as a last resort when a Gemini key is configured.

ROLE	MODEL	NOTES
Primary answer model	`openai/gpt-oss-120b` (Groq)	Reasoning hidden and set to low effort; temperature 0; 25 s timeout; automatic retry on 429 using the wait time Groq returns.
Fallback 1	`qwen/qwen3.8-27b` (Groq)	Reasoning hidden.
Fallback 2	`openai/gpt-oss-20b` (Groq)	Smaller, fastest.
Optional fallbacks	Gemini 3.x Flash family	Only when `GEMINI_API_KEY` is set; thinking disabled for speed.
Search embeddings	`BAAI/bge-small-en-v1.5` (local)	384 dimensions, ONNX on CPU, about 0.02 s per query.
Router (unclear messages only)	Same chain as the answer model	One-word reply, MANUAL or CHAT.

Every provider setting lives in `.env` (see `.env.example`), so the stack can be switched without code changes: `LLM_PROVIDER`, `GROQ_CHAT_MODEL`, `GROQ_CHAT_FALLBACK_MODELS`, `EMBEDDING_PROVIDER`, and their Gemini equivalents.

---

7. Safety and grounding

Because the subject is a medical device, the assistant is designed to be conservative. Five layers apply, in order:

1. **Input filter.** Regular expressions reject requests that try to operate the pump or treat a patient (start/stop the pump, change settings, program an infusion, dose the patient, deliver medication). The refusal returns a fixed scope warning and never reaches a model.
2. **Routing guard.** Device questions always go to retrieval. The conversational prompt explicitly instructs the model not to answer device, clinical, dosing, or alarm questions from its own knowledge and to redirect the user to ask a manual question instead.
3. **Grounded generation.** The answer prompt restricts the model to the supplied excerpts, forbids invented steps and clinical advice, and asks it to state when the excerpts are insufficient.
4. **Verification and abstention.** Answers without supporting evidence are replaced with an abstention. In the evaluation, the question the system could not find an answer for produced "the manual excerpts do not specify..." rather than a guess.

5. **Disclosure.** Every screen states that the software is not a medical device and is not for dosing, programming, or alarm-response decisions, and every answer lists its sources so the user can check the manual directly.

---

8. Backend API

The FastAPI application in `backend/main.py` is stateless: each request carries its own conversation history, so concurrent users never share state on the server. The agent, embedding model, and index are built once at startup in a background thread so the first question does not pay the set-up cost.

ENDPOINT	PURPOSE
`GET /health`	Liveness check.
`POST /chat`	One-shot answer with citations (JSON). Configuration or index problems return 503; model failures return 502 with the provider's message.
`POST /chat/stream`	Newline-delimited JSON events: `status` (thinking, searching, reading, refining), `token` (answer text as written), `done` (final answer, citations, structured sources with PDF page numbers), `error`. The final answer is authoritative, since the manual path may complete or replace the streamed text.
`GET /manual`	Whether the PDF is available, its page count, and the map from printed labels to PDF pages.
`GET /manual/pages/{n}.png`	One rendered page, cached per page.

```
{"type": "status", "stage": "searching"}
{"type": "status", "stage": "reading"}
{"type": "token", "text": "Meaning: Audio alarm failure – the audio alarm is
reported OFF"}
{"type": "token", "text": " while sensed ON, or ON while sensed OFF (High
priority).\n"}
{"type": "done", "answer": "Meaning: … Manual-listed action: Power off the
infuser. Replace pump.",
 "citations": ["plum_360_som_pn-ifu0000340_21658851.pdf (pages 182, 183, 184)",
"…"],
 "sources": [{"citation": "…", "pages": [182, 183, 184]}, …], "route":
"manual"}
```

Cross-origin access is allowed for the local Vite ports and for any origins listed in `CORS_ALLOWED_ORIGINS`.

9. Front end

The user interface is a React 18 single-page application built with Vite. It has no UI library; styling is hand-written CSS with a small token set (IBM Plex Sans and Mono, a single teal accent) so that the product page and the chat read as one product. Routing uses the URL hash: `#/` is the product guide and `#/assistant` the full-page chat; `#/assistant?q=…` opens the chat and asks that question.

Product guide

Hero with an original device illustration, key figures, six feature cards, four diagrams cropped from the manual, a five-step workflow, alarm priority levels, care, and specifications. Every fact shows its manual page label, and clicking it opens that page.

Chat drawer

Slides in from the right (full screen on phones) from the floating "Ask about Plum 360" button or any "Ask" link, which pre-asks the related question. Conversation state lives at the app root, so it survives moving between pages.

Streaming answers

Reads the NDJSON stream with the Fetch API. Status lines mirror the agent's progress; tokens are revealed at a steady typing pace that speeds up when behind; a blinking caret marks the live answer; Ask becomes Stop while generating.

Sources and page viewer

Sources show the manual's own page labels (for example 9-2 – 9-4). Clicking one opens a modal with the rendered page and arrows to step through all cited pages. Keyboard: Escape closes, arrow keys page.

Answer formatting

"Meaning" and "Manual-listed action" lines become labelled fields; bold and bullet points from the model are rendered; safety refusals appear in an amber notice.

Accessibility and robustness

Live regions for streaming text, focus management in the modal, reduced-motion support, and clear error states ("Could not reach the assistant API…") with the actual backend message.

A Streamlit app (`app.py`) remains as a minimal alternative interface that uses the same service module.

10. Evaluation

Quality is measured end to end by `scripts/evaluate_manual.py`, which sends 26 questions through the real pipeline (safety check, routing, retrieval, generation) and grades each one automatically. The question set covers four kinds of input.

TYPE	COUNT	CHECKS APPLIED
Manual questions (alarms, specifications, procedures, care)	20	*routing* = manual; *retrieval*: a cited chunk includes an expected manual page; *facts*: the answer contains every expected value (any accepted phrasing, after normalising spaces, dashes, and digit grouping)
Conversation (greeting, "what can you do", thanks)	3	*routing* = conversation; no citations
Off-topic ("How do I tune a guitar?")	1	*scope*: declined without citations
Unsafe ("Change pump settings...", "Dose the patient...")	2	*safety*: refused before the model

Results (Groq answers, local embeddings)

CHECK	PASSED	RATE
Routing (manual vs. conversation)	23 / 23	100%
Retrieval (expected page cited)	18 / 20	90%
Facts (answer contains expected values)	18 / 20	90%
Small talk answered without citations	3 / 3	100%
Off-topic declined	1 / 1	100%
Unsafe requests refused	2 / 2	100%
Overall	**24 / 26**	92%

Both failures are values in the dense specifications table on manual page 11-6. For the per-line minimum rate in concurrent delivery, the assistant stated that the excerpts did not specify it (a correct abstention, scored as a miss). For the delivery rate range, it cited a related programming page and gave an imprecise lower bound, which is a genuine inaccuracy. The same pipeline with Gemini embeddings scored 25 of 26; the local model is slightly weaker on dense tables.

The first evaluation run scored 81%. Analysis of the failures led to two improvements that are now part of the system: hybrid retrieval (section 4), which fixed misses on exact terms, and automatic retry on rate limits. Some checks were also loosened where the grader was stricter than correct answers warranted (for example, accepting "once per year" for "annual", or "9 999" for "9999"). The full per-question report is saved to

`evaluation/results/latest.md` on every run, and a `--rescore` flag re-grades saved answers without new model calls.

---

11. Automated tests

Sixty-five tests in 14 files run with `python -m unittest discover -s tests` in about 25 seconds. They use in-memory Chroma collections, deterministic fake embeddings, scripted fake models, a generated three-page PDF, and FastAPI's test client, so none of them needs an API key or network access.

TEST FILE	COVERS
`test_document_inspection`, `test_extraction_pipeline`, `test_chunking`	PDF inspection, page-aware extraction, token-bounded chunking with page mapping
`test_vector_store`, `test_keyword_search`, `test_embeddings`	Idempotent indexing, search metadata, BM25 tokenisation and ranking, rank fusion, provider and collection selection
`test_langgraph_agent`, `test_intent_routing`	Alarm-code routing, evidence narrowing, abstention, heuristic and LLM routing, history passed to conversational replies
`test_memory_and_safety`, `test_config`, `test_llm_providers`	Bounded conversation memory, input filter, settings validation, Groq/Gemini fallback chain order
`test_service_and_backend`, `test_manual_pages`	Per-caller history isolation, error kinds to HTTP status, NDJSON streaming events, page rendering, label mapping, manual endpoints
`test_app`	Streamlit smoke test

12. Technology stack

LAYER	TOOL	VERSION	ROLE IN THIS PROJECT
Language	Python	`3.12`	Core package, scripts, backend, tests
Agent framework	LangGraph	`1.2.20`	State graph with conditional edges, streaming of node updates and tokens
LLM tooling	LangChain (core, text-splitters)	`1.4.10`	Model abstraction, fallbacks, text splitting
LLM provider	Groq via `langchain-groq`	`1.1.3`	gpt-oss-120b, Qwen 3.8, gpt-oss-20b
Optional provider	Gemini via `langchain-google-genai`	`4.4.3`	Chat fallback and alternative embeddings
Embeddings	fastembed (ONNX Runtime)	`0.9.1`	Local `bge-small-en-v1.5` vectors
Vector database	Chroma via `langchain-chroma`	`1.5.9`	Persistent local collections
Keyword search	Custom BM25 (`keyword_search.py`)	—	Sparse ranking, rank fusion
Tokeniser	tiktoken	`0.14.0`	Token-measured chunking
PDF	PyMuPDF	`2.1`	Inspection, extraction, page and figure rendering
API	FastAPI + uvicorn	`0.162 / 0.54`	HTTP endpoints, NDJSON streaming, CORS
Validation / config	Pydantic, python-dotenv	`2.x`	Request models, `.env` settings
Front end	React + Vite	`18.3 / 5.4`	Product guide, chat, page viewer
Runtime	Node.js	`22`	Front-end tooling
Prototype UI	Streamlit	`1.67`	Alternative chat interface
Testing	unittest, FastAPI TestClient, httpx	—	65 offline tests

LAYER	TOOL	VERSION	ROLE IN THIS PROJECT
Version control	Git	–	Private data excluded via `.gitignore`

13. Repository structure

The repository tracks 70 files: about 4,500 lines of Python and 3,200 lines of front-end code. Everything derived from the manual stays outside version control.

```
Plum 360 Assistant/
├── plum360_assistant/        core package
│   ├── agent/                graph.py (LangGraph), router.py, tools.py,
state.py
│   ├── service.py            shared entry point: answer_question,
stream_answer
│   ├── vector_store.py       embeddings, Chroma, hybrid search_chunks
│   ├── keyword_search.py     BM25 index and rank fusion
│   ├── manual_pages.py       PDF page rendering, label map, figure crops
│   ├── llm.py                Groq / Gemini model chain with fallbacks
│   ├── safety.py             input filter and scope warning
│   ├── conversation_memory.py, config.py, knowledge_domains.py
├── backend/main.py           FastAPI app
├── frontend/src/             React app (App, ProductPage, ChatPage,
components/, useChat.js)
├── scripts/                  inspect, extract, chunk, index, search, export
figures, evaluate
├── evaluation/               questions.json, results/latest.md
├── tests/                    14 test modules
├── docs/                     progress notes
├── data/           (ignored) raw PDF, processed text, chroma_db, models
├── app.py                    Streamlit prototype
├── requirements.txt, .env.example, README.md
```

14. Design decisions and problems solved

The project was built iteratively. The decisions below shaped the final system; most came from a concrete problem met along the way.

PROBLEM OR QUESTION	DECISION	EFFECT
Two interfaces (Streamlit, API) gave different answers; the Streamlit one bypassed the agent.	One shared `service.py` used by both.	Identical behaviour everywhere; the agent is built once per process.
A module-level conversation memory mixed different users' chats on the server.	Stateless API; the client sends its own history each request.	Safe for concurrent users.
Every message was treated as a manual question; "how are you" returned "the manual does not provide...".	Intent router: rules for obvious cases, an LLM classifier for the rest, manual as the default.	Natural chat with no extra quota cost for most messages.
Gemini's free tier allowed about 20 chat requests per model per day and often returned 503/504.	Model fallback chain; later a switch to Groq for generation.	Reliable, fast answers (about 1 s to first token).
Groq has no embeddings API, and the Gemini embedding key was the last external dependency.	Local `bge-small` embeddings through fastembed, with a separate Chroma collection per provider.	No key or quota for search; manual text stays on the machine.
Vector search missed exact terms ("air trap", "hard limit", "rate range").	Hybrid retrieval: BM25 with word-pair terms, rank fusion that keeps each list's top result.	Top-3 retrieval rose from 9/16 to 18/20 on probe questions.
Reasoning models leaked their "thinking" into answers and were slow.	Hidden reasoning, low effort; Gemini thinking budget 0.	Clean answers; first text in about a second.
Groq's free tier caps tokens per minute (8,000).	Client retries using the wait time in the 429 response.	Bursts of questions wait briefly instead of failing.
Groq retired the Llama models first configured (404).	Model list queried from the account; defaults updated to available models.	Working defaults in `.env.example`.
The status line said "Searching the manual" even for small talk, and answers appeared all at once.	NDJSON streaming of status, tokens, and a final authoritative answer.	Honest progress display and a typing effect.
Users could not verify citations.	Render cited pages from the PDF; map printed labels to PDF pages.	One click from a source to the manual page.
Questions sent from the product page were cancelled instantly in development.	Removed an abort-on-unmount hook that React's development double-mount triggered.	Pre-asked questions run correctly.

PROBLEM OR QUESTION	DECISION	EFFECT
The evaluation grader rejected correct answers written with a narrow no-break space ("9 999").	Unicode normalisation of spaces, dashes, and digit grouping before comparison.	Grading measures content, not typography.
Manual text, PDF, and figures are copyrighted.	All derived data under git-ignored paths; the repository holds only code and the evaluation summary.	The project can be shared without redistributing the manual.

---

15. Limitations and future work

Current limitations

- **Dense tables.** Values in the specifications table (manual 11-6) are the weakest retrieval area; both evaluation misses are there.
- **Free-tier limits.** Groq allows about 8,000 tokens per minute, which is roughly two to three manual answers per minute before a short wait.
- **Single document, English only.** The pipeline is written for one manual; adding others requires re-indexing and a document filter.
- **Small evaluation set.** Twenty-six questions give a useful signal but not a statistically strong one, and the fact checks are keyword-based rather than judged by a model.
- **No authentication or persistence.** Conversations live in the browser tab; there is no login, logging, or analytics.
- **Not a medical device.** The system is a reference aid and has not been validated for clinical use.

Future work

- Table-aware chunking and a cross-encoder re-ranker to improve retrieval from specification tables.
- A larger evaluation set with an LLM-as-judge grader for answer quality, run automatically in continuous integration.
- Multi-document support (other pump models, service manuals) with per-document citations.
- User feedback on answers (helpful / not helpful) fed back into the evaluation set.
- Deployment with authentication, request logging, and a hosted front end.

16. How to run

```
# 1. Python environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env        # then set GROQ_API_KEY

# 2. Build the index once (PDF in data/raw/manuals/)
python -m scripts.extract_documents data\raw\manuals\<manual>.pdf --document-id
plum360_manual
python -m scripts.chunk_documents data/processed/plum360_manual.json
python -m scripts.index_chunks data/processed/plum360_manual_chunks.json
python -m scripts.export_manual_figures

# 3. Run (two terminals)
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --host 127.0.0.1 --port
8000 --reload
cd frontend; npm install; npm run dev        # open http://localhost:5173

# 4. Verify
python -m unittest discover -s tests
python -m scripts.evaluate_manual
```

17. Conclusion

The Plum 360 Assistant shows a complete, measured generative AI application built around one technical document. Its distinguishing features are grounding (answers are summaries of retrieved manual text, never general model knowledge), verifiability (page citations that open the actual page), routing (the system knows when to search and when to simply talk), and safety (unsafe requests are refused before any model call, and unsupported answers become abstentions).

Along the way the project worked through the practical problems of real LLM systems: provider rate limits and outages, retired models, retrieval misses on exact terminology, streaming, and keeping copyrighted source data private. Each of these led to a design change that is now tested, and the evaluation harness makes further changes measurable. The result passes 24 of 26 end-to-end questions and 65 automated tests, and runs with a single free API key.